

oystermapR

Command Reference & User Guide

Version 1.0.0 · April 2026

oystermapR predicts spatial habitat suitability for oyster growth from environmental survey data. Users supply sensor data (ADCP, CTD, bathymetric sonar, sidescan), specify a target species, and receive weighted suitability scores, a GeoTIFF heatmap for QGIS, contour lines, and a formatted report. Supports five European species with season-aware scoring and tidal-height correction.

```
library(oystermapR)

# 1. Load and merge your sensor data
adcp <- read_nortek_adcp("current.nc")
ctd <- read_generic_csv("ctd.csv")
survey <- merge_sensor_data(list(adcp, ctd))

# 2. (Optional) correct depths to chart datum
survey <- correct_to_chart_datum(survey, tidal_height_m = 1.4)

# 3. Predict suitability
result <- predict_oyster(survey, "ostrea_edulis",
                        output_geotiff = "kames_bay.tif")

# 4. Generate a report
generate_report(result, "report.html",
               title = "Kames Bay Assessment")
```

Contents

- 1. Data Ingestion** [read_nortek_adcp](#) · [estimate_chlorophyll_from_backscatter](#) · [classify_substrate_from_backscatter](#) · [read_generic_csv](#) · [read_soundings_xyz](#) · [read_sonar_tif](#)
- 2. Pre-processing** [stack_surveys](#) · [merge_sensor_data](#) · [qc_survey_data](#) · [auto_tidal_correct](#) · [add_intertidal_flag](#) · [interpolate_survey](#) · [correct_to_chart_datum](#) · [tidal_port_info](#)
- 3. Modelling** [predict_oyster](#) · [check_exclusions](#) · [score_locations](#) · [add_suitability_ci](#) · [smooth_suitability](#) · [detect_season](#) · [add_season_column](#)
- 4. Export** [export_geotiff](#) · [export_qml_style](#) · [export_contours](#)
- 5. Species Database** [get_species_tolerances](#) · [list_species](#)
- 6. Analysis** [compare_species](#) · [compare_surveys](#) · [composite_seasonal](#) · [score_settlement](#) · [score_disease_risk](#) · [project_suitability](#) · [identify_resilient_sites](#) · [analyse_connectivity](#) · [validate_against_records](#)
- 7. Planning** [assess_gear_feasibility](#) · [score_economic_viability](#)
- 8. Reporting** [generate_report](#)
- 9. Help** [oystermapR_help](#)
- A. QGIS Integration Guide** [Loading GeoTIFFs](#) · [Styling](#) · [Contours](#)

1 Data Ingestion

These functions read raw sensor output files and return a standardised dataframe suitable for merging and modelling. All functions attempt to auto-detect the coordinate reference system (WGS84 decimal degrees) and return columns named consistently: *lat*, *lon*, *depth*, *temperature*, *salinity*, *current_velocity*, etc.

read_nortek_adcp()

```
read_nortek_adcp(path, depth_mode = c('pressure', 'bin'), sidelobe_rm = TRUE, bed_shear = TRUE, verbose = TRUE)
```

Reads a Nortek Signature or Aquadopp ADCP export (NetCDF .nc or .mat). Computes depth-averaged current velocity, applies sidelobe contamination removal (bottom 6% of the water column), and optionally calculates bed shear stress ($\tau = \rho \cdot C_d \cdot U^2$). Returns one row per ensemble/ping location.

Key arguments:

path	Path to the .nc or .mat file.
depth_mode	pressure uses the pressure sensor depth; bin sums bin heights.
sidelobe_rm	Remove sidelobe-contaminated bins near the seabed (default TRUE).
bed_shear	Calculate bed shear stress column <i>bed_shear_stress_Pa</i> (default TRUE).
verbose	Print a summary of bins, ensembles and quality metrics (default TRUE).

```
adcp <- read_nortek_adcp("nortek_export.nc")
```

estimate_chlorophyll_from_backscatter()

```
estimate_chlorophyll_from_backscatter(df, backscatter_col, frequency_khz = 600, calibration = NULL, min_chl = 0.05, max_chl = 50, keep_raw = TRUE, verbose = TRUE)
```

Estimates chlorophyll-a concentration ($\mu\text{g/L}$) from ADCP acoustic backscatter strength using the empirical log-linear relationship: $Chl_a = 10^{(a \cdot Sv + b)}$. Useful when a fluorometer was not deployed alongside the ADCP. Built-in calibration constants (Deines 1999) are available for 300, 600, and 1200 kHz ADCPs. For best results, calibrate against concurrent water samples from the survey area. Adds a *chlorophyll_a* column directly, enabling chlorophyll scoring in subsequent *predict_oyster()* calls. Typical accuracy without calibration: factor 2–3 $\times$ (r^2 0.4–0.7); with site calibration: ≈ 1.3 – $1.5\times$ (r^2 0.7–0.9).

Key arguments:

df	Dataframe from <i>read_nortek_adcp()</i> or similar, containing a backscatter column.
backscatter_col	Name of the column containing volume backscatter strength (S_v , dB re 1 m^{-1}). For Nortek Signature outputs: typically <i>amp_mean_dB</i> .
frequency_khz	ADCP operating frequency in kHz (300, 600, or 1200). Selects built-in calibration constants.
calibration	Optional numeric vector $c(a, b)$ overriding built-in constants. Derive by fitting $\log_{10}(\text{measured_chl}) \sim \text{backscatter}$ from water samples.
min_chl / max_chl	Floor and ceiling for estimates (default 0.05–50 $\mu\text{g/L}$). Values outside this range are clipped; high values usually indicate turbidity artefacts rather than phytoplankton.

```
# Default calibration (Nortek Signature 300 kHz) adcp <- read_nortek_adcp('survey.csv') adcp
<- estimate_chlorophyll_from_backscatter( adcp, 'amp_mean_dB', frequency_khz = 300) # Custom
calibration from water samples fit <- lm(log10(sample_chl) ~ backscatter, data = grab_samples)
adcp <- estimate_chlorophyll_from_backscatter( adcp, 'amp_mean_dB', frequency_khz = 300,
calibration = c(coef(fit)[['backscatter']], coef(fit)[['(Intercept)']]))
```

■ Backscatter-to-chlorophyll conversion is affected by particle size, species composition, and suspended sediment. Always validate against at least a few fluorometer or grab-sample measurements before using estimated values for regulatory work.

classify_substrate_from_backscatter()

```
classify_substrate_from_backscatter(df, backscatter_col, is_sv = FALSE, bs_min = NULL, bs_max =
NULL, hard_thresh = 0.80, mixed_thresh = 0.50, soft_thresh = 0.20, output_col =
'substrate_hardness', verbose = TRUE)
```

Classifies seabed substrate hardness from near-seabed ADCP backscatter. Hard substrates (rock, cobble, shell hash) return more acoustic energy than soft substrates (mud, fine silt). The function normalises the backscatter column to a **hardness index [0-1]** and assigns one of four classes: Hard (≥ 0.8), Mixed (0.5-0.8), Soft (0.2-0.5), Very Soft (< 0.2). The resulting `substrate_hardness_index` column maps directly to the `substrate_hardness` scored variable in the tolerance spec.

Key arguments:

backscatter_col	Column name containing near-seabed backscatter values.
is_sv	If TRUE, treats values as Sv (dB, typically negative); harder = less negative. Default FALSE.
bs_min / bs_max	Backscatter values mapped to hardness 0 and 1. Default = 5th/95th percentile.
hard_thresh	Hardness index above which class = 'Hard' (default 0.80).
output_col	Base name for output columns; appends <code>_index</code> and <code>_class</code> (default 'substrate_hardness').

```
# Classify from ADCP near-seabed bin survey <- classify_substrate_from_backscatter( survey,
'bottom_backscatter_db', is_sv = TRUE) # Rename to match predict_oyster() expected column name
names(survey)[names(survey) == 'substrate_hardness_index'] <- 'substrate_hardness' result <-
predict_oyster(survey, 'ostrea_edulis')
```

■ Substrate classification assumes the backscatter column represents the seabed return, not mid-water scattering. Use the deepest clean bin from `read_nortek_adcp()` or a dedicated sidescan/sub-bottom profiler export.

read_generic_csv()

```
read_generic_csv(path, col_map = NULL, skip = 0, sep = ',', verbose = TRUE)
```

Reads any tabular CSV/TSV sensor export (CTD, YSI, turbidity probe, etc.) and renames columns to the oystermapper standard. Column mapping is auto-detected from common header synonyms; supply `col_map` to override.

Key arguments:

path	Path to the CSV or TSV file.
col_map	Named character vector mapping your column names to oystermapper names, e.g. <code>c(Temp_C = 'temperature', Sal_ppt = 'salinity')</code> .
skip	Number of header rows to skip before column names (default 0).
sep	Field separator character (default ',').

```
ctd <- read_generic_csv("ctd_cast.csv", col_map = c(Temp_C = "temperature", Sal_ppt =
"salinity"))
```

read_soundings_xyz()

```
read_soundings_xyz(path, lon_col = 'lon', lat_col = 'lat', depth_col = 'depth', crs = 4326,
verbose = TRUE)
```

Reads bathymetric point cloud data from a plain XYZ text file or CSV. Reprojects to WGS84 if a different CRS is supplied. The returned dataframe includes `lon`, `lat`, and `depth` columns.

Key arguments:

path	Path to the XYZ or CSV file.
lon_col	Column name for easting / longitude (default 'lon').
lat_col	Column name for northing / latitude (default 'lat').
depth_col	Column name for depth in metres (default 'depth').
crs	EPSG code of input coordinates (default 4326 = WGS84). Supply e.g. 27700 for British National Grid.

```
bathy <- read_soundings_xyz("lowrance_bathy.xyz", crs = 27700)
```

read_sonar_tif()

```
read_sonar_tif(path, band = 1, sample_n = NULL, verbose = TRUE)
```

Reads a single-band or multi-band GeoTIFF produced by a sonar processing package (e.g. Lowrance BioBase, SonarTRX) and converts raster cells to a point dataframe with columns `lon`, `lat`, and the band value. Useful for

importing substrate or backscatter layers.

Key arguments:

path	Path to the GeoTIFF (.tif / .tiff).
band	Raster band number to extract (default 1).
sample_n	Randomly subsample to at most this many points (default NULL = all).
<pre>substrate <- read_sonar_tif("sidescan_substrate.tif", band = 1)</pre>	

2 Pre-processing

Functions for assembling, spatially averaging, and tidally correcting data before modelling.

stack_surveys()

```
stack_surveys(..., spatial_res = 4, verbose = TRUE)
```

Combines multiple survey dataframes from the *same sensor type* (e.g. two ADCP deployments in overlapping areas) by row-binding and then spatially averaging overlapping cells on a regular grid. Use *merge_sensor_data()* to combine *different* sensor types.

Key arguments:

...	Two or more dataframes (or a list of dataframes) from the same sensor.
spatial_res	Grid resolution in metres for spatial averaging (default 4 m).
verbose	Print row counts before and after stacking (default TRUE).
<pre>adcp_all <- stack_surveys(adcp_flood, adcp_ebb, spatial_res = 4)</pre>	

merge_sensor_data()

```
merge_sensor_data(sensor_list, spatial_res = 4, verbose = TRUE)
```

Takes a named list of sensor dataframes (each already loaded and optionally stacked) and performs a spatial full-join: each dataframe is snapped to a common grid and merged by nearest grid cell. Missing values are preserved as NA rather than dropped, so all survey locations are retained.

Key arguments:

sensor_list	Named list of dataframes, e.g. <i>list(adcp=adcp_df, ctd=ctd_df)</i> . Also accepts a list of lists (auto-stacked per type).
spatial_res	Grid resolution in metres (default 4 m). Should match any prior stacking.
verbose	Print merge statistics (default TRUE).
<pre>survey <- merge_sensor_data(list(adcp = adcp_all, ctd = ctd_df, bathy = bathy_df))</pre>	

■ All input dataframes must contain lat and lon columns in WGS84 decimal degrees.

auto_tidal_correct()

```
auto_tidal_correct(df, datetime_col = 'date', depth_col = 'depth', max_port_dist_km = 75, keep_raw = TRUE, verbose = TRUE)
```

Fully automatic tidal height correction using an embedded 5-constituent harmonic model (M2, S2, N2, K1, O1). Finds the nearest reference port to the survey centroid (from 31 UK and European ports), predicts tidal height above Chart Datum for every survey timestamp, and calls *correct_to_chart_datum()* row-by-row. No internet connection or external data files required. Typical accuracy: ± 0.3 – 0.5 m. Adds a *tidal_height_pred_m* column showing the predicted height used.

Key arguments:

df	Survey dataframe with coordinate and datetime columns.
datetime_col	Column name containing survey timestamps. Accepts POSIXct or Date (date-only values are assumed noon UTC).

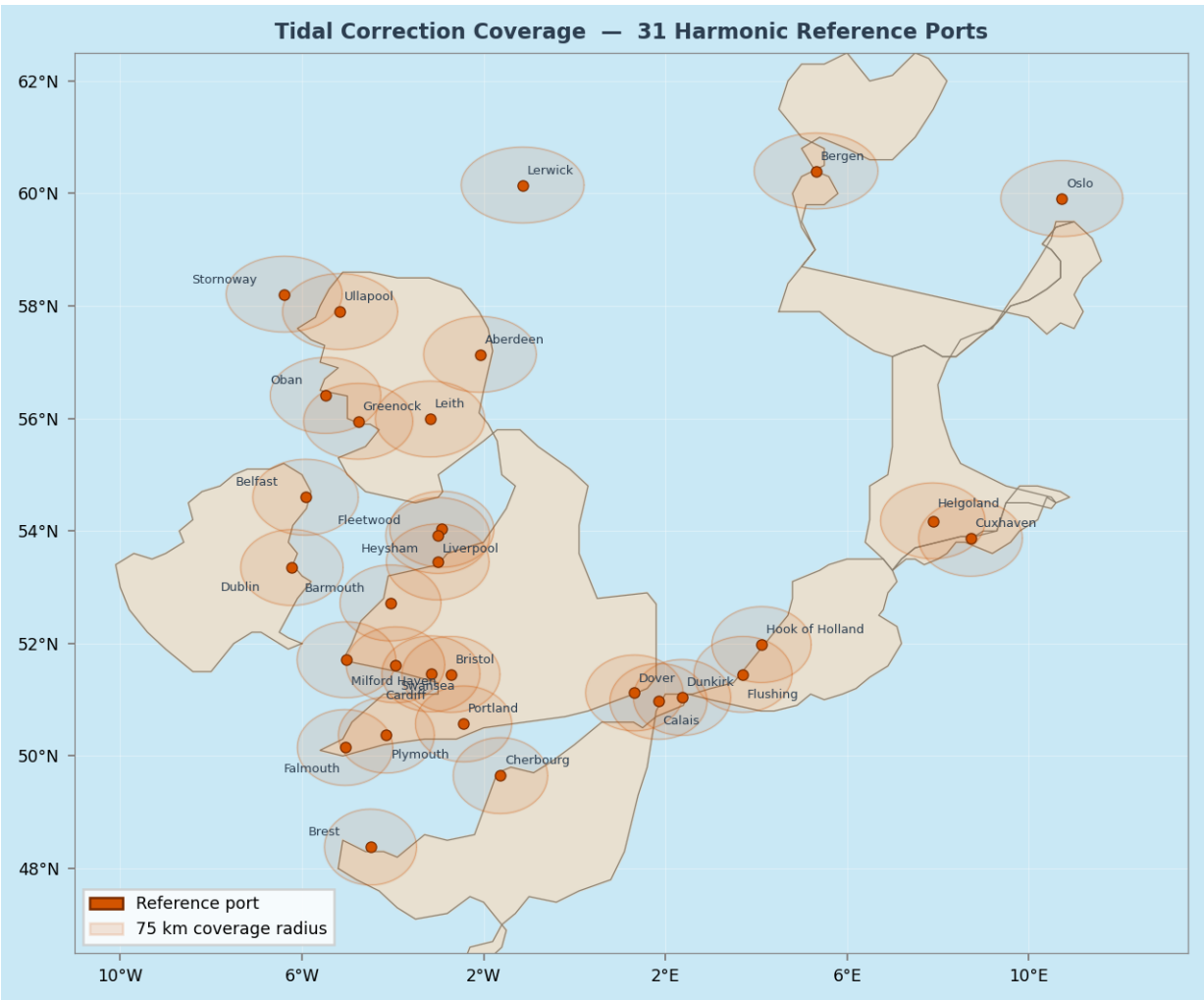
depth_col	Name of the depth column to correct (default 'depth').
max_port_dist_km	Maximum distance to nearest port in km; error if exceeded (default 75 km). Increase for remote surveys.
keep_raw	Preserve original depths in <i>depth_raw_m</i> (default TRUE).
verbose	Print nearest port name and predicted height range (default TRUE).

```
# Fully automatic – recommended for most surveys
survey <- auto_tidal_correct(survey,
  datetime_col = "date") # Extend search radius for remote areas
survey <- auto_tidal_correct(survey, max_port_dist_km = 120)
```

■ Coverage: 31 ports across UK, Ireland, France, Benelux, Germany, and Norway. See the tidal coverage map below. For surveys outside this region, use `correct_to_chart_datum()` with values from a local tide gauge.

Tidal Model Coverage — 31 Harmonic Reference Ports

Each orange dot marks a reference port with embedded M2, S2, N2, K1, O1 harmonic constituents (sources: UKHO ATT, SHOM, BSH, RWS, Kartverket). Shaded circles show the 75 km default search radius. `auto_tidal_correct()` selects the nearest port to your survey centroid automatically. Surveys outside all circles should use `correct_to_chart_datum()` with official tide table values.



add_intertidal_flag()

```
add_intertidal_flag(df, max_port_dist_km = 75, depth_col = 'depth', verbose = TRUE)
```

Classifies each survey location as *subtidal*, *intertidal*, or *supratidal* using the chart-datum depth and the local spring tidal range derived from the nearest harmonic reference port. Chart Datum is approximately Lowest Astronomical Tide (LAT), so: subtidal = always submerged (depth > 0 m CD); intertidal = exposed at some states of the tide (0 ≥ depth ≥ –MHWS); supratidal = above the highest tide (excluded from oyster scoring). MHWS is estimated as $Z_0 + 1.1 \times (M_2 + S_2 \text{ amplitudes})$ from the port constituents. The *intertidal_zone* flag is used by `score_locations()` to give *Magallana*

gigas a full depth score (1.0) at intertidal cells, reflecting its strongly intertidal ecology in NW European waters. Adds two columns: *intertidal_zone* and *mhws_above_cd_ref*.

Key arguments:

df	Survey dataframe. Must have <i>lat</i> , <i>lon</i> , and a chart-datum depth column. Typically output of <i>auto_tidal_correct()</i> .
max_port_dist_km	Maximum distance to nearest reference port (default 75 km). Falls back to a conservative 6 m MHWS window if exceeded.
depth_col	Name of the chart-datum depth column (default 'depth').

```
survey <- auto_tidal_correct(survey, datetime_col = 'date') survey <-
add_intertidal_flag(survey) # Check coverage table(survey$intertidal_zone) # subtidal: 1042
intertidal: 318 supratidal: 12
```

■ Run after *auto_tidal_correct()* or *correct_to_chart_datum()* so depths are referenced to CD before classification. For *M. gigas* surveys, intertidal coverage is often more productive than subtidal — inspect the breakdown before interpreting depth scores.

correct_to_chart_datum()

```
correct_to_chart_datum(df, tidal_height_m, depth_col = 'depth', datetime_col = NULL, keep_raw =
TRUE, verbose = TRUE)
```

Corrects survey depths from instantaneous water level to Chart Datum (CD) by adding the tidal height above CD at the time of each measurement. Formula: $depth_{CD} = depth_{surveyed} + tidal_height_above_CD$. Accepts a scalar constant or a per-row vector. Use *tidal_port_info()* to look up datum offsets for UK/European ports.

Key arguments:

df	Survey dataframe with a depth column.
tidal_height_m	Tidal height above CD in metres. Scalar for a constant correction; numeric vector of length <code>nrow(df)</code> for per-row values from a tide gauge.
depth_col	Name of the depth column to correct (default 'depth').
datetime_col	Optional name of a datetime column; used only for logging.
keep_raw	Preserve uncorrected depths in a new column <i>depth_raw_m</i> (default TRUE).

```
# Constant offset (tide gauge reading at start of survey) survey <-
correct_to_chart_datum(survey, tidal_height_m = 1.4) # Per-row tidal heights from a data
logger survey <- correct_to_chart_datum(survey, tidal_height_m = tide_vec)
```

tidal_port_info()

```
tidal_port_info(port = NULL)
```

Returns tidal datum reference data for 31 UK and European ports: MHWS, MLWS, mean spring tidal range, and Chart Datum below MSL. Call with no argument to print the full reference table.

Key arguments:

port	Port name (partial match, case-insensitive). E.g. 'oban', 'bristol', 'brest'. NULL prints the full table.
-------------	---

```
tidal_port_info() # show all ports tidal_port_info('oban') # single port lookup
```

qc_survey_data()

```
qc_survey_data(df, datetime_col = 'datetime', iqr_k = 3.0, max_gradients = NULL, apply_flags =
FALSE, verbose = TRUE)
```

Runs automated quality control on raw sensor data before modelling. Three checks per numeric column: (1) **range check** against physically plausible bounds for each variable (e.g. temperature -2 to 35°C); (2) **statistical outlier** detection using Tukey's outer fence ($iqr_k \times \text{IQR}$ from the median, default $k = 3$); (3) **temporal gradient** check when a datetime column is present — sequential change rates exceeding instrument-appropriate thresholds are flagged as instrument spikes. Additionally, **cross-variable sanity checks** flag physically implausible combinations (e.g. DO > 18 mg/L with temperature $< 5^{\circ}\text{C}$). Flagged values are marked in *qc_flag_<col>* columns but **not removed** by default, so the user retains control.

Key arguments:

datetime_col	Column name for datetime; enables gradient checks (default 'datetime'). NULL to skip.
iqr_k	IQR multiplier for outlier fence (default 3.0; use 1.5 for high-precision CTD data).
max_gradients	Named list of max change-per-second thresholds to override defaults.
apply_flags	If TRUE, replace flagged values with NA (default FALSE — flags only).

```
# Flag without removing survey_qc <- qc_survey_data(survey, datetime_col = 'timestamp')
subset(survey_qc, qc_status == 'fail') # inspect worst rows # Apply flags (replace outliers
with NA) before scoring survey_clean <- qc_survey_data(survey, apply_flags = TRUE) result <-
predict_oyster(survey_clean, 'ostrea_edulis')
```

■ *qc_status* = 'pass' (no flags), 'review' (1-2 flags), 'fail' (3+ flags). *qc_n_flags* counts total issues per row. Run this before *auto_tidal_correct()* or *interpolate_survey()*.

interpolate_survey()

```
interpolate_survey(survey, vars = NULL, resolution_m = 100, method = 'auto', kriging_min_n = 50,
idw_power = 2, idw_max_radius_m = NULL, verbose = TRUE)
```

Interpolates sparse point observations (CTD casts, ADCP profiles) to a regular grid before scoring. Returns a dataframe in the same format as the raw survey, ready to pass directly to *predict_oyster()*.

Method selection (applied per variable independently): **Ordinary kriging** (via *gstat*) is used when $\geq$ *kriging_min_n* observations are available — it is the Best Linear Unbiased Estimator (BLUE) and returns a kriging variance at each cell. **IDW** (Inverse Distance Weighting, $p=2$) is used as a fallback when observations are insufficient, *gstat* is not installed, or variogram fitting fails. Kriging variograms are auto-fitted using a Matérn model (most flexible for environmental data), falling back to spherical if needed.

Key arguments:

vars	Column names to interpolate. Default: all numeric columns except lat/lon.
resolution_m	Grid cell size in metres (default 100). Coarsen for sparse surveys.
method	'auto' (kriging if possible, else IDW), 'kriging' (force), or 'idw' (force).
kriging_min_n	Minimum observations to attempt kriging (default 50).
idw_power	IDW distance decay exponent (default 2).

```
# Auto selection – kriging for dense variables, IDW for sparse grid <-
interpolate_survey(survey, resolution_m = 100) result <- predict_oyster(grid, 'ostrea_edulis')
# Check which method was used unique(grid$interp_method_temperature) # 'kriging' or 'idw' #
Kriging variance – high = uncertain interpolation hist(grid$krige_var_temperature)
```

■ *Kriging requires gstat and sp packages: install.packages(c('gstat','sp')). IDW requires no extra packages. For surveys with < 50 casts, IDW is the practical choice.*

3 Modelling

The core pipeline: exclusion checking, variable scoring, and the main *predict_oyster()* wrapper that runs the full workflow.

predict_oyster()

```
predict_oyster(data, species, output_geotiff = NULL, resolution = 0.0002, buffer = 0.001,
idw_max_dist = NULL, contours = TRUE, verbose = TRUE)
```

Main entry point. Runs the complete suitability pipeline: (1) loads and validates data, (2) detects season from survey dates/lat, (3) applies hard exclusion criteria, (4) scores each location against species tolerances, (5) interpolates a suitability raster via IDW, (6) exports GeoTIFF and contour GeoPackage, (7) prints a console summary including the top 5 recommended introduction sites with patch radii. Returns a dataframe with one row per survey location.

Key arguments:

data	Dataframe or path to a CSV file. Must contain <i>lat</i> and <i>lon</i> .
species	Species key or partial name. See <i>list_species()</i> for valid keys.
output_geotiff	File path for the output GeoTIFF (e.g. 'survey.tif'). NULL or FALSE to skip raster export.
resolution	IDW raster resolution in decimal degrees (default 0.0002 ≈ 20 m).
buffer	Padding around the survey bounding box in decimal degrees (default 0.001).
idw_max_dist	Maximum IDW search radius in decimal degrees. NULL = auto (5× median nearest-neighbour spacing).
contours	Auto-export contour GeoPackage alongside the GeoTIFF (default TRUE).
verbose	Print pipeline progress and summary (default TRUE).

```
result <- predict_oyster( survey, "ostrea_edulis", output_geotiff = "kames_bay.tif",
resolution = 0.0002, contours = TRUE )
```

■ Output dataframe columns include: *suitability* [0–1], *suitability_class*, *excluded*, *limiting_factors*, *score_columns*, *season*.

check_exclusions()

```
check_exclusions(df, tolerances)
```

Applies hard exclusion criteria for the chosen species and returns a logical vector indicating which rows fail at least one hard limit (temperature outside lethal range, salinity below minimum, etc.). Excluded locations receive suitability = 0 and are not scored further. Called internally by *predict_oyster()*; exposed for inspection.

Key arguments:

df	Survey dataframe.
tolerances	Species tolerance list from <i>get_species_tolerances()</i> .

```
tol <- get_species_tolerances("ostrea_edulis") excl <- check_exclusions(survey, tol) sum(excl)
# number of excluded locations
```

score_locations()

```
score_locations(df, tolerances, verbose = FALSE)
```

Scores each non-excluded location against species tolerances using a rank-weighted mean (rank 1 = highest ecological importance). Handles seasonal temperature overrides automatically when a *season* column is present. For *Magallana gigas*, intertidal cells (requires prior *add_intertidal_flag()*) automatically receive a full depth score (1.0). Adds per-variable score columns, a *limiting_factors* column listing the 2–3 lowest-scoring variables, and a *data_completeness* column [0–1] showing the fraction of tolerance-spec variables that had real data and contributed to the weighted score.

Key arguments:

df	Survey dataframe (should include excluded column from <i>check_exclusions()</i>).
tolerances	Species tolerance list.

verbose	Print per-variable weight summary (default FALSE).
----------------	--

```
tol <- get_species_tolerances("ostrea_edulis") scored <- score_locations(survey, tol, verbose = TRUE) # Inspect data coverage quality summary(scored$data_completeness) # 0.4 = only 4/10 variables scored; treat cautiously
```

■ *data_completeness < 0.5 means fewer than half of the tolerance-spec variables had sensor data at that location. Suitability scores in those cells are less reliable and should be reported alongside the completeness value.*

add_suitability_ci()

```
add_suitability_ci(result, tolerances, n_boot = 200, uncertainty = NULL, seed = 42, verbose = FALSE)
```

Quantifies uncertainty in suitability scores via **bootstrap resampling**. Re-scores each row *n_boot* times with Gaussian noise added to every numeric variable, returning the 5th and 95th percentile scores as *suit_ci_lower* and *suit_ci_upper* (90% interval). The CI width reflects both measurement error and sensitivity of the scoring curves: locations near optimal-range boundaries will show wider intervals than those well inside the optimal zone. When CI columns are present, *generate_report()* adds uncertainty rings around map markers, scaled by CI width.

Key arguments:

result	Dataframe from <i>predict_oyster()</i> .
tolerances	Species tolerance list from <i>get_species_tolerances()</i> .
n_boot	Number of bootstrap iterations (default 200; increase to 500 for publication).
uncertainty	Named numeric vector of 1-sigma errors per variable, e.g. <i>c(temperature=0.3, salinity=0.5)</i> . Unspecified variables default to 2% of column mean.
seed	Random seed for reproducibility (default 42).

```
tol <- get_species_tolerances('ostrea_edulis') result <- add_suitability_ci(result, tol, uncertainty = c(temperature = 0.3, salinity = 0.5, depth = 1.0)) # Inspect uncertainty hist(result$suit_ci_width) # wide = uncertain # HTML report map rings scale with CI width automatically generate_report(result, 'report.html')
```

■ *n_boot = 200 takes ~5-15 seconds for 500 locations. Increase to 500 for final reporting; use 50 for quick exploratory runs. Adds columns: suit_ci_lower, suit_ci_upper, suit_ci_width.*

smooth_suitability()

```
smooth_suitability(result, bandwidth_m = 500, max_radius_m = NULL, smooth_excluded = FALSE, verbose = TRUE)
```

Applies a **Gaussian kernel smoother** to suitability scores, replacing each cell's value with a distance-weighted mean of all cells within *max_radius_m*. Weights decay as $\exp(-d^2 / 2\sigma^2)$ where $\sigma = \text{bandwidth_m}$. This reduces noise from single anomalous casts and produces spatially coherent GeoTIFF heatmaps. The original score is preserved in *suitability_raw*. Excluded cells are excluded from contributing to their neighbours' smoothed scores by default (hard barriers are preserved), but can be included by setting *smooth_excluded = TRUE*.

Key arguments:

bandwidth_m	Gaussian 1-sigma bandwidth in metres (default 500). Increase for sparser surveys; decrease to preserve fine spatial detail.
max_radius_m	Maximum search radius for neighbours (default 3 * bandwidth_m).
smooth_excluded	If TRUE, excluded cells contribute to neighbours' smoothed scores (default FALSE).

```
result <- smooth_suitability(result, bandwidth_m = 300) # Compare raw vs smoothed plot(result$suitability_raw, result$suitability, xlab = 'Raw', ylab = 'Smoothed', pch = 20) # Export smoothed surface as GeoTIFF export_geotiff(result, 'kames_bay_smooth.tif')
```

■ *Smoothing is computationally $O(n^2)$; for surveys with >5000 cells, consider reducing max_radius_m to limit neighbourhood size. suitability_class is reclassified from the smoothed score.*

detect_season()

```
detect_season(date, lat)
```

Returns the meteorological season ('winter', 'spring', 'summer', 'autumn') for a given date and latitude. Hemisphere-aware: southern hemisphere seasons are flipped. Returns NA if date or lat is NA.

Key arguments:

date	A Date or POSIXct object, or a character string parseable as a date.
lat	Latitude in decimal degrees (positive = N, negative = S).
<pre>detect_season(as.Date("2024-07-15"), lat = 56.4) # "summer"</pre>	

add_season_column()

```
add_season_column(df, date_col = 'date', lat_col = 'lat')
```

Vectorised wrapper around *detect_season()*. Adds a *season* column to a dataframe by applying season detection row-wise. Called automatically inside *predict_oyster()*.

Key arguments:

df	Survey dataframe.
date_col	Name of the date column (default 'date').
lat_col	Name of the latitude column (default 'lat').
<pre>survey <- add_season_column(survey, date_col = 'survey_date')</pre>	

4 Export

Functions for writing spatial output files for use in QGIS or other GIS software.

export_geotiff()

```
export_geotiff(df, output, resolution = 0.0002, buffer = 0.001, idw_power = 2, idw_max_dist = NULL, contours = FALSE, verbose = TRUE)
```

Interpolates scored point data to a regular raster using Inverse Distance Weighting (IDW) and writes a 4-band GeoTIFF. Band 1 = suitability score [0–1]; Band 2 = suitability class (1=Very Low ... 4=High); Band 3 = number of survey points within IDW radius; Band 4 = distance to nearest survey point in metres (uncertainty indicator). Called automatically by *predict_oyster()* when *output_geotiff* is set.

Key arguments:

df	Scored dataframe with <i>lat</i> , <i>lon</i> , <i>suitability</i> columns.
output	Output file path (.tif).
resolution	Raster cell size in decimal degrees (default 0.0002).
buffer	Bounding-box padding in decimal degrees (default 0.001).
idw_power	IDW power parameter (default 2).
idw_max_dist	Maximum search radius in decimal degrees (NULL = auto).
contours	Also write a contour GeoPackage (<i>output_contours.gpkg</i>) (default FALSE).
<pre>export_geotiff(result, "habitat_map.tif", resolution = 0.0002, contours = TRUE)</pre>	

■ Band 4 (distance to nearest point) can be used in QGIS to mask or fade areas far from actual survey coverage. High values indicate extrapolation.

export_qml_style()

```
export_qml_style(output = 'oystermapper_style.qml', palette = c('YlOrRd', 'viridis', 'terrain'))
```

Writes a QGIS Layer Style (.qml) file that applies the oystermapper colour ramp to Band 1 of your GeoTIFF. Load in QGIS via *Layer Properties* → *Style* → *Load Style*.

Key arguments:

output	Output .qml file path.
palette	Colour palette: 'YlOrRd' (default), 'viridis', or 'terrain'.

```
export_qml_style("kames_bay_style.qml", palette = "viridis")
```

export_contours()

```
export_contours(tif_path, output = NULL, levels = c(0.2, 0.45, 0.70), smooth = TRUE)
```

Generates suitability contour lines from a GeoTIFF produced by *export_geotiff()* and saves them as a GeoPackage (.gpkg). Default contour levels correspond to the Very Low/Low/Moderate/High class boundaries.

Key arguments:

tif_path	Path to the suitability GeoTIFF.
output	Output .gpkg path. NULL = auto-name alongside the GeoTIFF.
levels	Numeric vector of suitability values at which to draw contours.
smooth	Apply Gaussian smoothing before contouring (default TRUE).

```
export_contours("kames_bay.tif", levels = c(0.2, 0.45, 0.70))
```

5 Species Database

Key	Latin name	Common name	Data quality
ostrea_edulis	Ostrea edulis	European Flat Oyster	✓ High
magallana_gigas	Magallana gigas	Pacific Oyster	✓ High
crassostrea_angulata	Crassostrea angulata	Portuguese Oyster	~ Medium
ostrea_stentina	Ostrea stentina	Denticulate Flat Oyster	! Low

Data quality: ✓ High = peer-reviewed primary sources; ~ Medium = reliable secondary sources; ! Low = estimated from related species / limited literature.

get_species_tolerances()

get_species_tolerances(species)

Returns the full tolerance parameter list for a species. The returned list includes *latin_name*, *common_name*, *region*, *data_quality*, *exclusions* (hard limits), *scored_factors* (weighted variables with optimal/tolerance ranges), and *seasonal_overrides* (season-specific temperature ranges). Accepts the species key, latin name, or common name (partial match).

Key arguments:

species Species key (e.g. 'ostrea_edulis'), latin name (e.g. 'Ostrea edulis'), or common name (e.g. 'flat oyster'). Case-insensitive.

```
tol <- get_species_tolerances("ostrea_edulis") tol$exclusions$temperature # hard temperature
limits tol$scored_factors$salinity # scoring parameters
```

list_species()

list_species(verbose = TRUE)

Prints a formatted table of all species in the database, showing their key, latin name, common name, native region, and data quality tier. Returns a dataframe invisibly.

Key arguments:

verbose Print the formatted table to the console (default TRUE).

```
list_species()
```

6 Analysis

compare_species()

compare_species(data, species = NULL, output_dir = NULL, min_data_quality = 'low', verbose = TRUE)

Runs *predict_oyster()* for every specified species against the same survey dataset and returns a wide comparison table — one row per survey location, one suitability column per species. Automatically applies a **competition penalty** when both *Magallana gigas* and *Ostrea edulis* are compared: where *M. gigas* scores High, *O. edulis* suitability is multiplied by 0.80; where *M. gigas* scores Moderate, by 0.92. The raw *suit_ostrea_edulis* column is preserved; the adjusted values appear in *suit_ostrea_edulis_competition_adj*.

Key arguments:

data Dataframe or CSV path (same format as *predict_oyster*).

species Character vector of species keys. NULL = all species in database.

output_dir Directory to write per-species GeoTIFF heatmaps. NULL = no export.

min_data_quality 'low' (all species), 'medium', or 'high' only.

verbose Print per-species summary table (default TRUE).

```
# Compare all species comp <- compare_species(survey) # View competition-adjusted O. edulis
scores hist(comp$suit_ostrea_edulis_competition_adj) # Sites suitable for 2+ species (robust
sites) robust <- subset(comp, n_species_high >= 2)
```

■ *Output columns: suit_ [0-1], class_, best_species, best_suitability, n_species_high, suit_ostrea_edulis_competition_adj (when M. gigas is included).*

validate_against_records()

```
validate_against_records(predicted, records, presence_col = 'presence', match_radius_deg = 0.002,
plot = TRUE, verbose = TRUE)
```

Compares suitability predictions from *predict_oyster()* against a known presence/absence dataset (e.g. OSPAR, NBN Atlas, ICES WGOS records) to quantify model discrimination. Spatially matches each record to the nearest prediction cell within *match_radius_deg*, then computes: AUC (area under the ROC curve), TSS (True Skill Statistic = sensitivity + specificity – 1), optimal threshold via Youden's J, sensitivity, specificity, F1, and Brier score. No external packages required — ROC is computed from first principles. Returns a named list; pass the result directly to *generate_report()* to embed a full interactive ROC chart in the HTML report.

Key arguments:

predicted	Dataframe from <i>predict_oyster()</i> with <i>lat</i> , <i>lon</i> , <i>suitability</i> .
records	Presence/absence dataframe with coordinate columns and a presence column (1/0 or TRUE/FALSE).
presence_col	Name of the presence/absence column in <i>records</i> (default 'presence').
match_radius_deg	Radius for matching records to prediction cells (default 0.002° ≈ 220 m).
plot	Print an ASCII ROC curve to the console (default TRUE).

```
# Load known O. edulis records from NBN Atlas CSV export records <-
read.csv('nbn_ostrea_edulis.csv') result <- predict_oyster(survey, 'ostrea_edulis') # Validate
val <- validate_against_records(result, records) val$auc # 0.78 = good discrimination val$tss
# 0.52 = useful skill # Embed in HTML report generate_report(result, 'report.html', validation
= val)
```

■ *Interpretation guide: AUC >= 0.8 = good; 0.6-0.8 = moderate; < 0.6 = near-random. TSS > 0.4 is generally considered useful predictive skill. The optimal threshold (Youden's J) is the suitability cutoff that best separates presences from absences in your validation dataset.*

compare_surveys()

```
compare_surveys(surveys, cell_deg = 0.001, stable_threshold = 0.05, verbose = TRUE)
```

Compares suitability scores across multiple surveys or monitoring years. Takes a **named list** of *predict_oyster()* result dataframes and produces: (1) a per-survey summary table with mean suitability, class breakdown, and data completeness; (2) a spatial dataframe joining all surveys on shared lat/lon cells with a per-cell *trend_slope* (linear regression of suitability over survey index); and (3) a pairwise change table showing the fraction of cells that improved, degraded, or remained stable between consecutive surveys.

Key arguments:

surveys	Named list of dataframes from <i>predict_oyster()</i> .
cell_deg	Spatial rounding tolerance in decimal degrees for matching cells (default 0.001 ≈ 111 m).
stable_threshold	Absolute suitability change below which a cell is 'stable' (default 0.05).
verbose	Print comparison summary (default TRUE).

```
r2022 <- predict_oyster(s2022, 'ostrea_edulis') r2023 <- predict_oyster(s2023,
'ostrea_edulis') r2024 <- predict_oyster(s2024, 'ostrea_edulis') comp <- compare_surveys(
list('2022' = r2022, '2023' = r2023, '2024' = r2024)) comp$summary # mean scores by year
comp$change # year-on-year change table comp$spatial # spatial grid with trend_slope column
```

■ *Returns a named list: \$summary, \$spatial, \$change. trend_slope > 0 indicates improving habitat; < 0 indicates degradation.*

composite_seasonal()

```
composite_seasonal(surveys, method = 'min', prob_threshold = 0.5, cell_deg = 0.001, verbose =
TRUE)
```

Combines multi-season *predict_oyster()* results into a single year-round composite suitability score. Locations are only truly suitable if conditions are adequate in every season; **method = 'min'** enforces this conservatively. Three composite methods are available: *min* (worst-season score — recommended for site selection), *mean* (average across seasons — good for monitoring), and *prob* (fraction of seasons above *prob_threshold*).

Key arguments:

surveys	Named list of dataframes from <i>predict_oyster()</i> .
method	'min', 'mean', or 'prob' (default 'min').
prob_threshold	Suitability threshold for method = 'prob' (default 0.5).
cell_deg	Spatial rounding tolerance in decimal degrees (default 0.001).

```
spring <- predict_oyster(s_spring, 'ostrea_edulis') summer <- predict_oyster(s_summer,
'ostrea_edulis') autumn <- predict_oyster(s_autumn, 'ostrea_edulis') comp <-
composite_seasonal( list(spring=spring, summer=summer, autumn=autumn), method = 'min')
generate_report(comp, 'year_round_report.html')
```

■ Output includes *suit_range* column (max - min across seasons) — high values flag locations with strongly seasonal suitability.

score_settlement()

```
score_settlement(survey, species = 'ostrea_edulis', verbose = TRUE)
```

Scores survey locations for **spat settlement suitability** using a separate, tighter tolerance specification than the adult habitat model. Larvae are more sensitive to turbidity, require minimum current velocity for delivery to the site, and need hard substrate with biofilm for attachment. Supported species: *ostrea_edulis*, *magallana_gigas*. Compare the output against adult *predict_oyster()* results to identify locations suitable for both adult survival **and** natural recruitment.

Key arguments:

survey	Dataframe of survey measurements (same format as <i>predict_oyster</i>).
species	Species key: 'ostrea_edulis' or 'magallana_gigas'.
verbose	Print settlement scoring summary (default TRUE).

```
adult <- predict_oyster(survey, 'ostrea_edulis') settlement <- score_settlement(survey,
'ostrea_edulis') # Merge and identify dual-suitable sites merged <- merge(adult, settlement[,
c('lat','lon','settlement_suitability')], by = c('lat','lon')) merged$dual_suitable <-
merged$suitability >= 0.6 & merged$settlement_suitability >= 0.6
```

■ Settlement columns are prefixed *settle_score_** to avoid collision with adult *score_** columns when merged. Key differentiators from adult spec: turbidity hard limit 20 NTU (vs 50), minimum current 0.05 m/s required, temperature 16-22°C optimum.

score_disease_risk()

```
score_disease_risk(result, species = 'ostrea_edulis', known_sites = NULL, temp_col =
'temperature', salinity_col = 'salinity', turbidity_col = 'turbidity', verbose = TRUE)
```

Assesses environmental risk from two key bivalve pathogens: **Bonamia ostreae** (*O. edulis*) and **OsHV-1 microvariant** (*M. gigas*). Risk is temperature-driven: *Bonamia* transmission peaks 15–20°C and is negligible below 8°C; OsHV-1 mortality events consistently occur above 16°C. Salinity modulates *Bonamia* risk; turbidity amplifies OsHV-1 risk by increasing host stress. A proximity multiplier is applied when *known_sites* (confirmed infected locations) are supplied. Risk scores are kept separate from ecological suitability — a High suitability site with Critical disease risk is a regulatory red flag.

Key arguments:

species	'ostrea_edulis' (<i>Bonamia</i>) or 'magallana_gigas' (OsHV-1).
known_sites	Dataframe of confirmed infected sites (lat/lon). Applies proximity multiplier.
temp_col	Temperature column name (default 'temperature').
salinity_col	Salinity column (used for <i>Bonamia</i> modulation, default 'salinity').
turbidity_col	Turbidity column (used for OsHV-1 amplification, default 'turbidity').

```
result <- predict_oyster(survey, 'ostrea_edulis') infected <- data.frame(lat=c(55.8,56.1),
lon=c(-5.2,-5.4)) result <- score_disease_risk(result, 'ostrea_edulis', known_sites =
infected) # Sites combining high suitability with high disease risk flagged <- subset(result,
```

```
suitability_class == 'High' & disease_risk_class %in% c('High','Critical'))
```

■ Risk classes: Low / Moderate / High / Critical. References: Culloty & Mulcahy (2007) for Bonamia; Garcia et al. (2011) for OsHV-1.

project_suitability()

```
project_suitability(result, tolerances, scenarios = c('rcp26','rcp45','rcp85'), custom_deltas = NULL, horizon = '2050s', verbose = TRUE)
```

Re-scores survey locations under future climate scenarios by applying temperature and salinity deltas to the observed data and re-running the suitability model. Built-in scenarios align with UKCP18 marine projections: **rcp26** (+0.5°C SST), **rcp45** (+1.2°C, -0.2 PSU), **rcp85** (+2.5°C, -0.5 PSU). Returns a named list with one result dataframe per scenario plus a comparative summary table showing mean change and fraction of cells improving or declining.

Key arguments:

scenarios	Built-in scenario keys: 'rcp26', 'rcp45', 'rcp85'.
custom_deltas	Named list of custom scenarios, e.g. <code>list(my_sc = c(temperature=1.8, salinity=-0.3))</code> .
horizon	Time horizon label for output (default '2050s').

```
tol <- get_species_tolerances('ostrea_edulis') proj <- project_suitability(result, tol)
proj$summary # mean scores and change by scenario # Cells that stay Moderate+ under all
scenarios resilient <- identify_resilient_sites(proj, result)
```

■ `identify_resilient_sites()` filters to cells that maintain the minimum class under every scenario. Use these as priority targets for long-term lease or restoration investment.

analyse_connectivity()

```
analyse_connectivity(result, min_suitability = 0.5, gap_m = 500, min_patch_cells = 3, verbose = TRUE)
```

Identifies spatially connected patches of suitable habitat using a union-find graph algorithm. Two cells are connected if within `gap_m` metres (default 500 m $\approx$ larval dispersal over one tidal cycle). Isolated cells and small patches are flagged as high-risk for recruitment failure and genetic bottlenecks. Returns patch IDs, patch area, and a connectivity class per cell. The highest-scoring cell in each patch is flagged as *is_hub* — the recommended introduction point for that patch.

Key arguments:

min_suitability	Score threshold to include a cell (default 0.5 = Moderate+).
gap_m	Max gap between connected cells in metres (default 500).
min_patch_cells	Patches smaller than this are classified 'isolated' (default 3).

```
result <- analyse_connectivity(result, gap_m = 500) table(result$connectivity_class) # Large
well-connected patches = best restoration targets hubs <- subset(result, is_hub &
connectivity_class == 'large') # Export patch map as GeoTIFF export_geotiff(result,
'patches.tif')
```

■ `patch_id` maps to distinct colours in QGIS for easy patch visualisation. `patch_area_km2` is estimated from the median nearest-neighbour distance between survey cells. No external packages required.

7 Planning

assess_gear_feasibility()

```
assess_gear_feasibility(result, gear_types = c('longline','bottom_culture',
'intertidal_rack','raft','restoration_reef'), depth_col = 'depth', current_col =
'current_velocity', substrate_col = 'substrate_hardness_class', wave_col = NULL, verbose = TRUE)
```

Evaluates which aquaculture and restoration gear types are physically viable at each survey location based on depth, current velocity, substrate, and wave height (if available). **This function is part of the planning module** — it is aimed at commercial operators and restoration practitioners assessing site practicality, and is kept separate from the core ecological model to avoid cluttering conservation-focused outputs. Five gear types are assessed: **longline**

(suspended, 4–25 m), **bottom culture** (bags/cages on seabed), **intertidal rack/trestle**, **raft/pontoon** (needs sheltered water), and **restoration reef** (no harvest, widest physical tolerance). Returns a binary feasibility flag and a continuous score [0-1] per gear type.

Key arguments:

gear_types	Gear types to assess. Default: all five.
depth_col	Depth below CD column (default <i>'depth'</i>).
current_col	Current velocity column (default <i>'current_velocity'</i>).
substrate_col	Substrate class column from <i>classify_substrate_from_backscatter()</i> .
wave_col	Significant wave height column Hs (default NULL, skipped if absent).

```

result <- predict_oyster(survey, 'ostrea_edulis') result <- assess_gear_feasibility(result) #
Longline-feasible + High suitability subset(result, gear_longline_feasible & suitability_class
== 'High') # Restoration reef sites (no licence needed in many areas) subset(result,
gear_restoration_reef_feasible)

```

■ Gear specs are based on: FAO (2004) oyster aquaculture site guidelines, SAMS/Marine Scotland shellfish farm benchmarking, and Falconer et al. (2013) suspended culture criteria. *best_gear* and *n_gear_options* columns are added automatically.

score_economic_viability()

```
score_economic_viability(result, harbours = NULL, target = 'restoration', verbose = TRUE)
```

Combines ecological suitability, gear feasibility, patch area, and site accessibility into a composite **economic viability index** [0-1]. This is a **heuristic ranking tool** — not a financial model — but it consistently ranks sites for shortlisting before detailed business case development. Two target modes adjust the component weights: **restoration** (default) prioritises ecological suitability (45%) and connected patch area (25%); **aquaculture** increases gear feasibility (30%) and harbour access (20%) weights for commercial deployment planning. When a *harbours* table is supplied, distance to the nearest licensed landing point is scored on a log-scale decay (0 km = 1.0, 50 km = 0).

Key arguments:

harbours	Dataframe of harbour/landing locations with lat/lon. NULL = access score = 0.5 (neutral).
target	<i>'restoration'</i> or <i>'aquaculture'</i> — adjusts component weights.

```

result <- predict_oyster(survey, 'ostrea_edulis') result <- assess_gear_feasibility(result)
result <- analyse_connectivity(result) harbours <- data.frame( lat = c(55.865, 55.875), lon =
c(-5.425, -5.300)) # Restoration ranking result <- score_economic_viability(result, harbours =
harbours, target = 'restoration') # Aquaculture ranking result <-
score_economic_viability(result, harbours = harbours, target = 'aquaculture') subset(result,
viability_class == 'Excellent')

```

■ Viability classes: Poor / Fair / Good / Excellent. Requires *assess_gear_feasibility()* to have been run first for gear scoring. Patch area scoring requires *analyse_connectivity()* output (*patch_area_km2* column).

8 Reporting

generate_report()

```
generate_report(result, output = 'oyster_report.html', title = 'Oyster Suitability Assessment',
author = '', species = 'ostrea_edulis', top_n = 5, validation = NULL, comparison = NULL, open =
TRUE)
```

Renders a fully interactive HTML report (or PDF) from the output of *predict_oyster()*. The HTML report uses Plotly for interactive charts and custom CSS with a sticky sidebar navigation. Sections include: executive summary stat cards, suitability histogram (coloured by class), class breakdown bar chart, per-variable mean scores, data completeness histogram, most common limiting factors, top introduction sites table, optional competition adjustment overlay (when *comparison* is supplied), optional validation metrics with interactive ROC curve (when *validation* is supplied), and a methodology reference table.

Key arguments:

result	Dataframe returned by <i>predict_oyster()</i> .
---------------	---

output	Output file path. Extension determines format: <i>.html</i> (recommended — interactive Plotly charts, no extra system software) or <i>.pdf</i> (requires LaTeX / tinytex).
title	Report title shown on the cover page.
author	Author name(s) shown on the cover page.
species	Species key used in the original <i>predict_oyster()</i> call.
top_n	Number of top introduction sites to include (default 5).
validation	Named list from <i>validate_against_records()</i> . When supplied, adds a Validation section with AUC, TSS, Brier score, confusion matrix metrics, and an interactive ROC curve.
comparison	Dataframe from <i>compare_species()</i> . When supplied, adds a Competition Adjustment section showing the <i>M. gigas</i> competitive pressure penalty on <i>O. edulis</i> scores.
open	Open the report in the default viewer after rendering (default TRUE).

```
# Standard HTML report (recommended) generate_report(result, "kames_bay.html", title = "Kames
Bay Habitat Assessment", author = "T. Tucker") # With validation metrics embedded val <-
validate_against_records(result, nbn_records) generate_report(result,
"kames_bay_validated.html", validation = val) # With competition adjustment section comp <-
compare_species(survey, species = c("ostrea_edulis", "magallana_gigas"))
generate_report(result, "kames_bay_full.html", validation = val, comparison = comp) # PDF
report – requires tinytex or a LaTeX installation # install.packages('tinytex');
tinytex::install_tinytex() generate_report(result, "kames_bay.pdf")
```

■ *HTML reports require plotly and scales: install.packages(c('plotly','scales')). All reports require rmarkdown and knitr: install.packages(c('rmarkdown','knitr')). PDF output additionally requires a LaTeX installation.*

9 Help

oystermapper_help()

`oystermapper_help(topic = NULL)`

Prints formatted usage instructions to the console. Call with no argument for a list of available topics; pass a topic name for detailed guidance on that workflow step.

Key arguments:

topic	One of: 'sensors', 'merge', 'predict', 'qgis', 'species'. NULL lists all topics.
--------------	--

```
oystermapper_help() # list all topics oystermapper_help('sensors') # sensor loading guide
oystermapper_help('predict') # modelling workflow oystermapper_help('qgis') # QGIS integration
guide
```

A QGIS Integration Guide

Loading the GeoTIFF

1. Open QGIS and navigate to **Layer** → **Add Layer** → **Add Raster Layer**.
2. Browse to your *.tif* file and click **Add**.
3. The GeoTIFF has 4 bands. By default QGIS displays Band 1 (suitability score [0–1]).

Applying the Colour Style

1. Export the QGIS style file from R: `export_qml_style('style.qml')`.
2. In QGIS, right-click the layer → **Properties** → **Symbolology** → **Style** → **Load Style**.
3. Select the *.qml* file. The layer will update immediately.

Understanding the 4 Bands

Band	Content	Range	Suggested use
1	Suitability score	0 – 1	Primary display layer (colour ramp)
2	Suitability class	1 – 4	Categorical labels: 1=Very Low, 4=High
3	Survey point density	0 – n	Data coverage overview
4	Distance to nearest survey pt	0 – ∞ m	Mask or fade interpolated areas

Loading Contour Lines

Contour lines are saved as a GeoPackage (*.gpkg*) alongside the GeoTIFF when `contours = TRUE` is used (the default). Load via **Layer** → **Add Layer** → **Add Vector Layer**. Each contour feature has a *level* attribute corresponding to the suitability value (0.20 = Low/Very Low boundary; 0.45 = Moderate; 0.70 = High).

Masking with Band 4 (Uncertainty)

Band 4 contains the distance (in metres) from each raster cell to the nearest actual survey point. Cells with large Band 4 values are interpolations far from real data and should be treated cautiously. In QGIS you can create a transparency mask: right-click the layer → **Properties** → **Transparency** → **Add values from display**, or use the Raster Calculator to set Band 1 = NoData where Band 4 > 200.

Suitability Class Boundaries

Class	Score range	Interpretation
High	≥ 0.70	Conditions closely match species optima across all scored variables
Moderate	0.45 – 0.70	Suitable with some sub-optimal conditions; viable for introduction
Low	0.20 – 0.45	Marginal habitat; consider as secondary / future monitoring sites
Very Low	< 0.20	Conditions significantly outside species tolerances
Excluded	0 (hard)	One or more variables breached hard exclusion limits